

RINGKASAN PROYEK

Payment Sandbox Application

Technical Assessment — Backend (Golang)

<https://github.com/Trickster-ID/payment-sandbox>

Dibuat: 05 May 2026

1. Gambaran Umum Proyek

Payment Sandbox adalah aplikasi simulasi transaksi pembayaran yang dibangun sebagai bagian dari Technical Assessment Software Engineering. Aplikasi ini mensimulasikan alur pembayaran nyata — mulai dari pembuatan invoice, halaman pembayaran publik, simulasi status pembayaran, hingga proses refund — tanpa integrasi ke payment gateway pihak ketiga.

Peran yang didukung sistem:

- Merchant — membuat invoice dan menerima pembayaran (simulasi)
 - End User (Payer) — membayar invoice melalui payment link
 - Admin — mengoperasikan sandbox: mengubah status pembayaran, refund, dan top-up
-

2. Fitur Utama yang Diimplementasikan

Autentikasi & Otorisasi

- Registrasi user dengan validasi email dan hash password (bcrypt via `golang.org/x/crypto/bcrypt`)
- Login menghasilkan JWT token dengan expiry yang dapat dikonfigurasi
- Role-based access control: MERCHANT vs ADMIN via middleware
- OAuth2 Authorization Server — Authorization Code, Client Credentials, dan Refresh Token grant types
- Scope system: read, write, admin dengan auto-approve untuk first-party clients

Merchant Wallet

- Merchant dapat melihat saldo wallet simulasi
- Pembuatan top-up request dengan status PENDING
- Admin dapat mengubah status top-up menjadi SUCCESS atau FAILED

- Saldo merchant bertambah secara atomik (DB transaction) jika top-up SUCCESS
- Daftar riwayat top-up dengan paginasi

Manajemen Invoice

- Merchant dapat membuat invoice dengan nomor unik otomatis
- Filter daftar invoice berdasarkan status, tanggal, dan parameter lain
- Payment link token yang random dan unik untuk setiap invoice
- Halaman pembayaran publik dapat diakses tanpa autentikasi via token
- Simulasi status EXPIRED ketika due_date terlewat

Simulasi Pembayaran

- End user memilih metode pembayaran: WALLET / VA_DUMMY / EWALLET_DUMMY
- Sistem membuat payment_intent dengan status PENDING
- Admin mengubah status ke SUCCESS atau FAILED
- Jika SUCCESS: status invoice otomatis menjadi PAID (atomic via DB transaction)
- State machine yang ketat untuk transisi status

Simulasi Refund

- Merchant mengajukan permintaan refund untuk invoice PAID
- Alur: REQUESTED → APPROVED/REJECTED → SUCCESS/FAILED
- Admin approve/reject lalu proses hasil refund
- Saldo merchant berkurang secara atomik jika refund SUCCESS

Admin Dashboard & Statistik

- Statistik total invoice, total PAID/FAILED/EXPIRED
 - Total nominal pembayaran dan refund
 - Filter berdasarkan merchant dan rentang tanggal
 - Panel manajemen payment intent, refund, dan top-up
-

3. Daftar Peningkatan (Improvements)

Berikut adalah seluruh peningkatan yang dilakukan melebihi persyaratan dasar assessment, dikelompokkan berdasarkan area teknis.

3.1 Arsitektur & Clean Code

- Migrasi dari arsitektur monolitik (sandbox module) ke modular clean architecture per domain
- Modul aktif: users, invoice, payment, refund, wallet, admin, oauth2, ledger, reconciliation, saga
- Penerapan layered architecture yang konsisten: handler → service → repository di setiap modul

- Dependency Injection menggunakan Google Wire — semua dependensi di-wire secara compile-time
- Konvensi penamaan interface seragam dengan prefix I* (IUserService, IInvoiceRepository, dll.)
- Entitas domain dipindahkan ke module-local (app/modules/<module>/models/entity/) — tidak ada shared store
- Split route registration: satu RegisterRoutes per modul API package
- Akses database menggunakan database/sql standard library + driver pgx/v5 (raw SQL, tanpa ORM) untuk kontrol query yang eksplisit

3.2 Modul Tambahan di Luar Spesifikasi Dasar

- Ledger module — pencatatan double-entry bookkeeping untuk audit transaksi keuangan
- Reconciliation module — proses rekonsiliasi antara invoice, payment, dan ledger
- Saga module — orchestrator pattern untuk transaksi terdistribusi dengan recovery
- OAuth2 module — Authorization Server lengkap (lihat 3.6)

3.3 Testing — Unit & Integrasi

- Unit test table-driven untuk semua service layer (users, invoice, payment, refund, wallet, admin)
- Unit test table-driven untuk semua handler layer dengan mock service menggunakan Mockery
- Coverage service layer (snapshot terbaru, 05 May 2026): users 95.8%, invoice 100%, payment 100%, refund 100%, wallet 100%, admin 100%, oauth2 ~91.7%, ledger 100%, merchants 100%, reconciliation 100%, saga orchestrator 100%
- Integration test dengan database nyata (PostgreSQL) mencakup alur merchant, admin, refund, dan negative cases
- Negative integration test: token tidak valid, role salah, input tidak valid, transisi status ilegal
- Standar assertions: testify/require untuk precondition, testify/assert untuk perilaku
- Perintah make test-integration, make verify-batch11, make coverage-services tersedia

3.4 Mock & Testability

- Setup Mockery dengan .mockery.yaml untuk semua interface repository dan service
- Mock digenerate per-modul di direktori mocks/ masing-masing
- Handler difactor untuk depend pada service interface — bukan concrete struct
- Satu perintah make mock untuk regenerasi semua mock secara konsisten
- Penghapusan handler-level interface duplikat — mock cukup dari service package

3.5 Shared Utilities

- Shared pagination utility (app/shared/pagination) dengan sanitizer dan default yang aman
- Shared validator utility (app/shared/validator) untuk validasi email, amount, dan tanggal
- Shared error extraction dan response envelope dengan unit test mandiri

- Response envelope standar (Envelope{data, meta, error}) digunakan di seluruh handler
- Request ID middleware dengan propagasi header X-Request-ID ke setiap response

3.6 Journey Logging (MongoDB)

- Infrastruktur journey logging ke MongoDB untuk audit trail event kritis
- Integrasi di semua transaction handler: wallet, invoice, payment intent, refund
- Best-effort pattern: jika Mongo tidak tersedia, fallback ke no-op logger — sistem tidak crash
- Konfigurasi via env: MONGO_URI, MONGO_DB_NAME, MONGO_COLLECTION, MONGO_JOURNEY_ENABLE
- Docker Compose menginclude service MongoDB dengan healthcheck yang benar

3.7 OAuth2 Authorization Server

- Implementasi OAuth2 penuh: Authorization Code, Client Credentials, Refresh Token grant types
- Self-service client registration — merchant dapat mendaftarkan aplikasi OAuth2 mereka sendiri
- Consent screen: auto-approve untuk first-party, halaman consent untuk third-party
- Scope granular: read, write, admin
- JWT access token (stateless) + opaque refresh token (disimpan di DB)
- Role claims diinclude dalam semua jenis grant token
- Tabel DB baru: oauth2_clients, oauth2_authorization_codes, oauth2_refresh_tokens, oauth2_consent

3.8 Dokumentasi API (Swagger/OpenAPI)

- Anotasi Swagger lengkap di semua handler (users, invoice, payment, refund, wallet, admin)
- Response schema konkret dengan tipe, contoh, dan enum — bukan generic map[string]interface{}
- Swagger metadata PaginationMeta sebagai tipe berdiri sendiri
- Dokumen digenerate via make swag dan tersedia di /swagger/index.html
- Parity review antara Swagger spec dan runtime routes — perbedaan /healthz vs /ping diperbaiki

3.9 Performance & Query Optimization

- EXPLAIN (ANALYZE, BUFFERS) dijalankan untuk semua query kritis
- Indeks database ditambahkan pada kolom filter yang sering dipakai
- Penghindaran N+1 query di semua list endpoint
- K6 performance test dengan profil: smoke, baseline, stress, soak
- Target API response time ≤ 300ms tercapai pada endpoint utama
- HTML report dari K6 tersimpan di docs/k6/
- GitHub Actions workflow untuk K6 smoke test (.github/workflows/perf-smoke.yml)

3.10 Keamanan (Security)

- CORS middleware dengan konfigurasi origin yang dapat dikontrol
- JWT expiry wajib; refresh token opaque untuk OAuth2
- Payment link token random dan unik — tidak bisa ditebak
- Password di-hash menggunakan bcrypt (golang.org/x/crypto/bcrypt)
- Tidak ada informasi sensitif yang terekspos ke response publik
- Dokumen ISO 27001-aligned security controls di [docs/iso/](#)
- Backup-restore continuity drill terotomasi: make drill-backup-restore
- CI workflow untuk ISO readiness verification: make verify-iso-ci

3.11 DevOps & Tooling

- Docker Compose lengkap: PostgreSQL, MongoDB, pgweb (UI query DB), aplikasi Go
 - Makefile dengan target terstruktur: build, test, swag, mock, coverage, verify-batch*
 - Dockerfile multi-stage untuk build image Go yang efisien
 - CI GitHub Actions: ISO verification dan K6 smoke test otomatis pada push/PR
 - Skrip make verify-batch11 menggabungkan test suite + route parity + query-plan checks
 - pgweb service untuk inspeksi database langsung dari browser
-

4. Tech Stack (Backend)

Bahasa & Framework

- **Bahasa:** Go 1.26
- **Framework HTTP:** Gin (github.com/gin-gonic/gin)
- **Dependency Injection:** Google Wire

Database & Persistensi

- **Database Utama:** PostgreSQL
- **Database Driver:** pgx/v5 ([jackc/pgx](#)) via database/sql standard library
- **Akses Data:** Raw SQL queries (tanpa ORM) — kontrol penuh atas query
- **Database Audit Log:** MongoDB (go.mongodb.org/mongo-driver)

Authentication & Security

- **JWT:** [golang-jwt/jwt/v5](#)
- **Password Hashing:** [bcrypt](https://golang.org/x/crypto/bcrypt) (golang.org/x/crypto/bcrypt)
- **UUID:** [google/uuid](#)

Testing & Mocking

- **Test Library:** testify (require + assert)
- **Mock Generator:** Mockery ([vektra/mockery v2](#))
- **SQL Mock:** [go-sqlmock](#)

Tooling & Dokumentasi

- **API Docs:** Swagger via swaggo/swag + gin-swagger
- **Performance Test:** K6 (Grafana K6)
- **Logging:** rs/zerolog
- **Config:** spf13/viper + johogodotenv

Infrastructure

- **Containerisasi:** Docker + Docker Compose
- **CI/CD:** GitHub Actions (iso-verification, perf-smoke)
- **DB GUI:** pgweb

Catatan: Frontend (React/React Native) tercantum dalam project requirement namun pada repository ini hanya backend yang diimplementasikan. Rencana frontend (Next.js) tersedia di .agents/frontend-generation-plan.md sebagai dokumen perencanaan — belum dieksekusi.

5. Struktur Proyek Backend

```
app/  
  cmd/           ← Entry point, router, DI wire  
  config/        ← Konfigurasi aplikasi  
  middleware/    ← Auth, request-ID, CORS  
  shared/        ← Shared utilities (response, errors, pagination,  
                  validator, journeylog)  
  
modules/  
  users/         ← Registrasi & login + JWT  
  oauth2/        ← OAuth2 Authorization Server  
  invoice/       ← Invoice CRUD + payment link  
  payment/       ← Payment intent lifecycle  
  refund/        ← Refund lifecycle  
  wallet/        ← Top-up + balance  
  admin/         ← Dashboard statistik + admin actions  
  ledger/        ← Double-entry bookkeeping  
  reconciliation/ ← Rekonsiliasi transaksi  
  saga/          ← Saga orchestrator + recovery  
  
docs/           ← Swagger spec, performance report, ISO docs, K6  
misc/           ← Scripts verify, ops, init DB  
.agents/        ← AI generation plans & progress trackers  
.github/workflows/ ← CI: iso-verification, perf-smoke
```

6. Catatan Akhir

Proyek backend ini telah melebihi persyaratan minimum assessment dengan menambahkan OAuth2 Authorization Server, journey logging ke MongoDB, ledger double-entry, saga

orchestrator, test coverage tinggi di semua layer, ISO security controls, dan performance testing dengan K6. Semua komponen dapat dijalankan dengan satu perintah docker-compose up dan diverifikasi dengan make verify-batch11.

Semua test lulus (go test ./...) dan route regression test berjalan di CI.

7. Laporan Coverage Unit Test

Laporan ini dihasilkan dari file coverage.out menggunakan go tool cover -func. Coverage diukur pada tanggal 05 May 2026 (commit terbaru: c3777ba — try to fix mount on postgre). Coverage total keseluruhan kode (termasuk mocks, api routes, dan docs) adalah 41.6%. Angka ini rendah karena file mocks/, api/routes.go, dan docs/docs.go tidak dicakup oleh unit test — ini adalah hal yang normal karena file-file tersebut adalah generated code dan route registration.

Coverage Total (all packages): 41.6%

7.1 Coverage Per Layer (Rata-Rata)

Berikut adalah rata-rata coverage per lapisan arsitektur, tidak termasuk file mocks/:

- 98.4% — Handler Layer (semua modul)
- 97.1% — Service Layer (semua modul)
- 90.0% — Repository Layer (semua modul)

7.2 Coverage Per Modul

Tabel di bawah menunjukkan coverage per modul, dikelompokkan berdasarkan layer:

Modul	Handler	Service	Repository
admin	100.0%	100.0%	100.0%
invoice	100.0%	100.0%	100.0%
ledger	100.0%	100.0%	0.0% (hanya integrasi)
merchants	100.0%	100.0%	100.0%
oauth2	~94.2%	~91.7%	~93.1%
payment	100.0%	100.0%	~90.6%
reconciliation	—	100.0%	—
refund	100.0%	100.0%	~93.0%
saga	—	~70.0%	—
users	100.0%	~95.8%	100.0%
wallet	~99.4%	100.0%	~99.7%

7.3 Area Coverage Rendah & Penjelasan

Beberapa area memiliki coverage rendah dengan alasan yang valid:

- ledger/repositories (0.0%): Repository ledger hanya diuji melalui integration test dengan database nyata (PostgreSQL), bukan unit test dengan go-sqlmock. Ini adalah keputusan desain yang disengaja karena query ledger bersifat kompleks.
- saga/services/recovery.go (40.0%): Fungsi recovery saga mencakup skenario partial-failure yang sulit disimulasikan dalam unit test tanpa infrastruktur lengkap.
- payment/sagas/ (~64.0% rata-rata): Beberapa branch dalam payment saga constructor dan rollback function hanya dapat diuji dalam konteks integrasi penuh.
- api/routes.go (0.0%): File route registration tidak dicakup karena merupakan glue code — diverifikasi lewat integration test (router_test.go).
- shared/locking/redis_lock.go (0.0%): Redis lock Acquire/Release tidak di-unit-test karena memerlukan koneksi Redis nyata.

7.4 Coverage Shared Utilities

Package shared/ memiliki coverage tinggi karena sepenuhnya diuji secara mandiri:

- shared/idempotency (Claim, Fetch, Complete): 100.0%
- shared/pagination (Parse): 100.0%
- shared/response (JSON, OK, Created, Fail, FailFromError): 100.0%
- shared/validator (IsEmail, IsPositiveAmount, ParseRFC3339, IsTodayOrFuture, IsISO4217Code): 100.0%
- shared/locking/optimistic (CheckedExec): 100.0%
- middleware (Auth, CORS, RequestID, RoleGuard): ~95.0%

7.5 Coverage cmd/ & Infrastructure

- app/cmd/app.go (NewApp): 100.0%
- app/cmd/router.go (RegisterRoutes): 97.3%
- app/cmd/main.go: 0.0% (entry-point — tidak di-unit-test, diverifikasi lewat integrasi)
- app/cmd/wire_gen.go: 0.0% (generated code dari Google Wire)
- app/config/config.go: 100.0%

7.6 Kesimpulan Coverage

Coverage unit test proyek ini sangat baik pada semua layer yang relevan. Business logic (service layer: 97.1% rata-rata) dan HTTP handler (98.4%) memiliki coverage hampir penuh. Coverage repository layer (90.0%) sedikit lebih rendah karena beberapa modul mengutamakan integration test untuk validasi SQL query nyata ke PostgreSQL. File-file yang memiliki coverage 0% (mocks/, api/routes.go, wire_gen.go, docs/docs.go) adalah generated code atau glue code yang tidak membutuhkan unit test terpisah.

Catatan: Untuk menjalankan ulang coverage, gunakan: make coverage-services atau go test -coverprofile=coverage.out ./... && go tool cover -func=coverage.out
